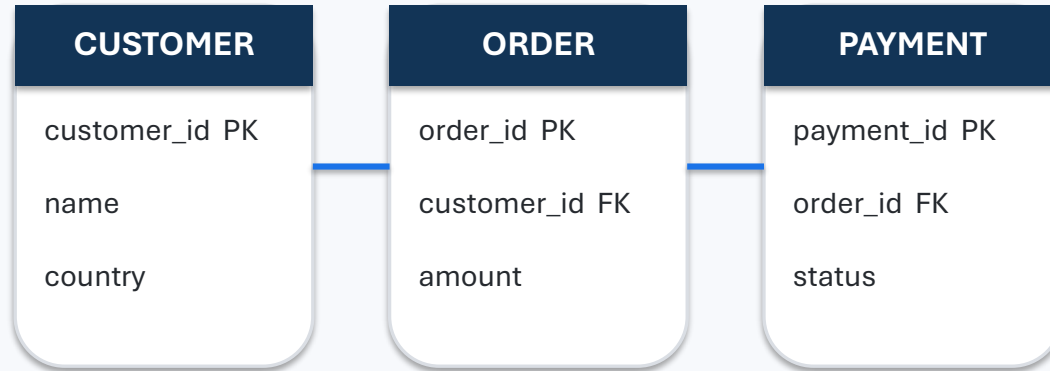


Stockage de la donnée : bases relationnelles (SQL)

Des tables structurées reliées par des clés, avec un schéma explicite et des transactions.

Modèle



SQL décrit les données et leurs relations.
Les jointures reconnectent les tables au moment de la requête.

Forces

- transactions ACID
- jointures riches
- contraintes / intégrité
- SQL standardisé

Limites

- schéma plus rigide
- scale-out plus complexe
- moins adapté à certains patterns distribués

Exemples

PostgreSQL

MySQL

SQL Server

Oracle

À privilégier lorsque cohérence transactionnelle, relations et requêtes SQL complexes sont prioritaires.

Stockage de la donnée : bases documentaires

Chaque enregistrement est un document autonome, souvent JSON/BSON, avec un schéma flexible.

Un document peut contenir des structures imbriquées

```
{
  "customer_id": 42,
  "name": "Iris",
  "address": {
    "city": "Paris",
    "country": "FR"
  },
  "tags": ["premium",
"mobile"]
}
```

COLLECTION

document

document

document

Forces

- schéma flexible
- données imbriquées
- évolution rapide
- lecture par document

Limites

- duplication fréquente
- relations moins naturelles
- cohérence multi-documents à concevoir

Exemples

MongoDB

CouchDB

Le document regroupe les données lues ensemble : on accepte souvent un peu de duplication pour simplifier les accès.

Stockage de la donnée : bases Wide-Column

RowKey + familles de colonnes + qualifieurs : un modèle distribué, sparse et orienté patterns d'accès.

Structure logique

RowKey : app#payment-api#20260115#000001

Famille : event

event_type
status_code
latency_ms

Famille : tech

source_team
host
service

Forces

- scale-out natif
- modèle sparse
- accès rapide par RowKey
- très gros volumes

Limites

- requêtes ad-hoc limitées
- design centré accès
- joins non naturels

Exemples

HBase

Cassandra

À ne pas confondre avec une base analytique colonne comme ClickHouse.

Wide-Column = données organisées par clé et familles de colonnes. HBase et Cassandra ne sont pas des entrepôts OLAP colonne comme ClickHouse.

Stockage de la donnée : bases Key-Value

Le modèle le plus direct : une clé unique permet de retrouver rapidement une valeur.

Accès principal : GET(key) / PUT(key, value)

session:84721

{"user":42,"ttl":900}

cart:1002

[A12,B07,C31]

feature:dark_mode

true

Forces

- très faible latence
- modèle simple
- scale-out efficace
- très bon pour accès par clé

Limites

- requêtes secondaires limitées
- relations complexes difficiles
- modélisation guidée par la clé

Exemples

DynamoDB

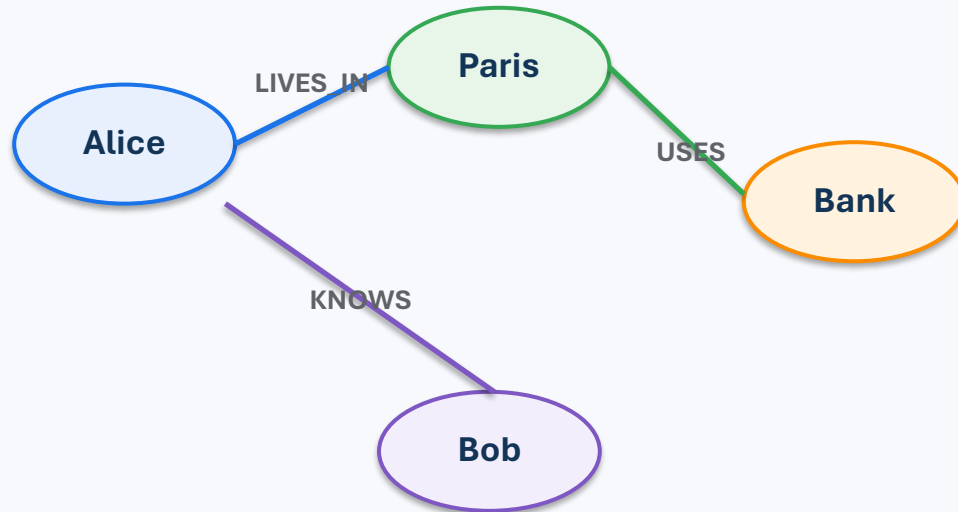
Redis

Excellent lorsque l'application connaît déjà la clé de recherche. Beaucoup moins adapté aux relations ou explorations ad-hoc.

Stockage de la donnée : bases graphe

Les relations deviennent des objets de premier plan : nœuds, relations et propriétés.

Modéliser les relations directement



Forces

- traversées rapides
- relations complexes
- modèle intuitif pour réseaux
- recommandations / fraude

Limites

- pas idéal pour tous les workloads
- modèle spécifique
- agrégations massives parfois moins naturelles

Exemples

Neo4j

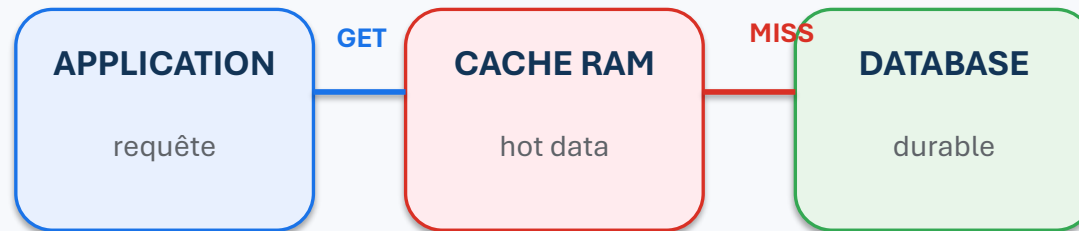
Amazon
Neptune

À privilégier lorsque la question porte sur les relations et les chemins : fraude, réseaux, dépendances, recommandations.

Stockage de la donnée : cache et bases en mémoire

Les données les plus sollicitées restent en RAM pour réduire fortement la latence d'accès.

Pattern classique : cache devant le stockage durable



CACHE HIT = réponse depuis la RAM

CACHE MISS = lecture du stockage durable

Forces

- latence très faible
- soulage la base principale
- idéal sessions / compteurs / cache

Limites

- RAM coûteuse
- éviction possible
- durabilité à configurer selon le produit

Exemples

Redis

Memcached

Un cache accélère l'accès aux données chaudes. Il ne remplace pas automatiquement un stockage durable de référence.